

PRÁCTICA 3

Sistema interactivo de navegación geográfica del viaje a Erebor

Alejandro Martín Vázquez 43224406T

GRUPO 5

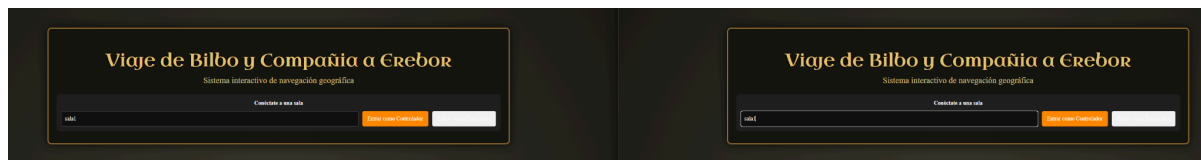
1. Idea general del proyecto

Sistema multimedia colaborativo en tiempo real

El proyecto consiste en una aplicación web que permite que dos usuarios visualicen contenido multimedia de forma sincronizada a través de Internet. Ambos participantes comparten una misma sesión donde las acciones realizadas por uno de ellos se reflejan automáticamente en el otro cliente.

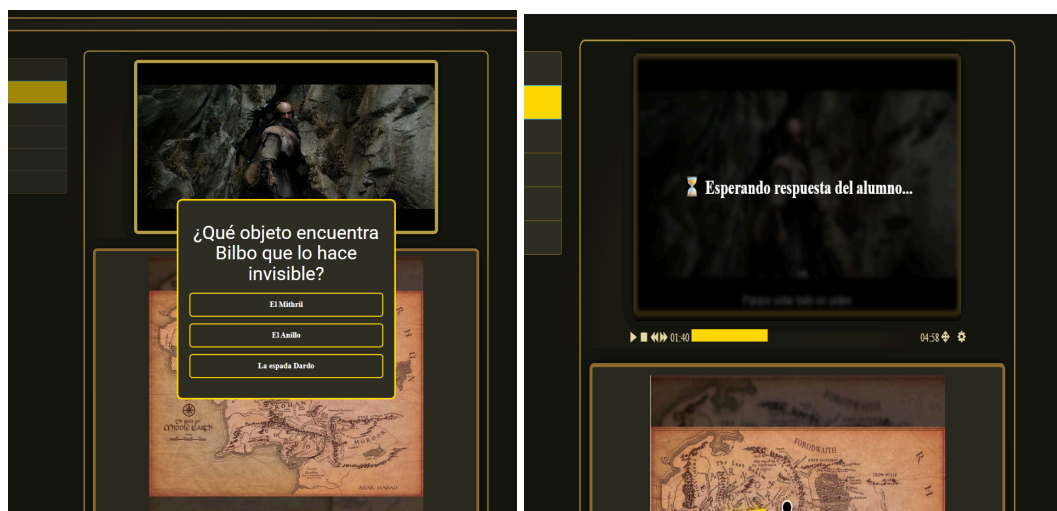
Sincronización entre usuarios A y B

La aplicación establece una relación colaborativa entre dos participantes. El usuario A actúa como controlador principal mientras que el usuario B recibe en tiempo real todas las acciones ejecutadas sobre el contenido multimedia.



Integración de vídeo interactivo y cuestionarios

Durante la reproducción pueden aparecer preguntas que interrumpen temporalmente el vídeo para comprobar la comprensión del contenido. Las respuestas se gestionan de forma sincronizada para ambos participantes.



Comunicación mediante WebRTC y RTCDataChannel

La comunicación entre usuarios utiliza la tecnología WebRTC para establecer conexiones directas P2P. A través de RTCDataChannel se transmiten comandos, estados y eventos sin necesidad de pasar por el servidor.

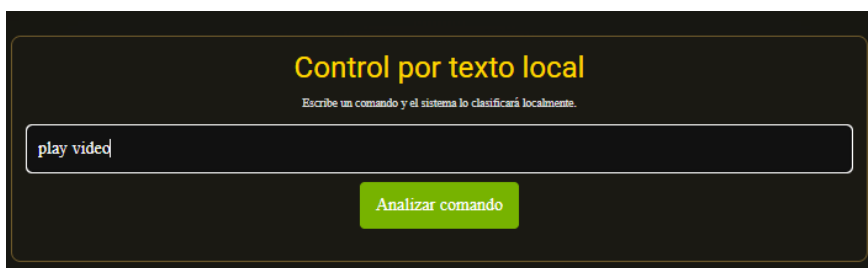
```
Creando offer...
roomInfo <div id="roomInfo" style="margin-top:10px; color:#ddd;">Sala: sala1 · Rol: Controlador. Sala lista.</div>
Offer enviada
ICE enviado
WS recibido: peer-joined
Hay un peer en la sala: peer-joined
Creando offer...
WS recibido: room-ready
Sala lista para comenzar
WS recibido: request-offer
Solicitud de oferta recibida
Creando offer...
Offer enviada
WS recibido: answer
Answer recibida
WS recibido: ice-candidate
ICE recibido
Canal abierto
```

Sistema de salas colaborativas

Los usuarios pueden crear o unirse a salas virtuales que permiten aislar las comunicaciones y mantener sesiones independientes.

Control inteligente mediante Transformers.js

Se incorpora un sistema de interpretación de lenguaje natural que permite ejecutar determinadas acciones multimedia mediante comandos escritos por el usuario.



2. Objetivos del sistema

- Implementar comunicación P2P mediante WebRTC
 - Utilizar WebSocket como sistema de señalización
 - Sincronizar reproducción multimedia entre usuarios
 - Diferenciar roles de controlador y espectador
 - Incorporar RTCDataChannel para intercambio de comandos
 - Permitir respuesta sincronizada de cuestionarios
 - Implementar un sistema de salas colaborativas
 - Integrar comandos interactivos mediante lenguaje natural
 - Mostrar el estado y progreso compartido entre usuarios
-

3. Arquitectura del sistema

Arquitectura híbrida cliente-servidor

La aplicación combina un servidor WebSocket encargado de la señalización con conexiones directas WebRTC entre los clientes. Este enfoque reduce la carga del servidor y mejora la velocidad de comunicación.

WebSocket para señalización

El servidor WebSocket interviene únicamente durante la fase inicial de conexión para intercambiar los mensajes necesarios para establecer la comunicación P2P.

RTCPeerConnection para comunicación P2P

Una vez completada la señalización, los clientes establecen una conexión directa mediante RTCPeerConnection.

RTCDataChannel para intercambio de eventos

Los eventos de reproducción, pausas, cambios de tiempo y respuestas de cuestionarios se transmiten a través del canal de datos.

Sistema de sincronización multimedia

Se mantiene la coherencia temporal entre ambos clientes mediante mensajes de actualización de estado.

Gestión de roles y salas

La arquitectura contempla la asignación automática de roles y el aislamiento de sesiones mediante identificadores de sala.

Transformers.js para procesamiento de comandos inteligentes

El modelo de lenguaje se ejecuta directamente en el navegador, evitando dependencias externas y mejorando la privacidad.

4. Estructura funcional

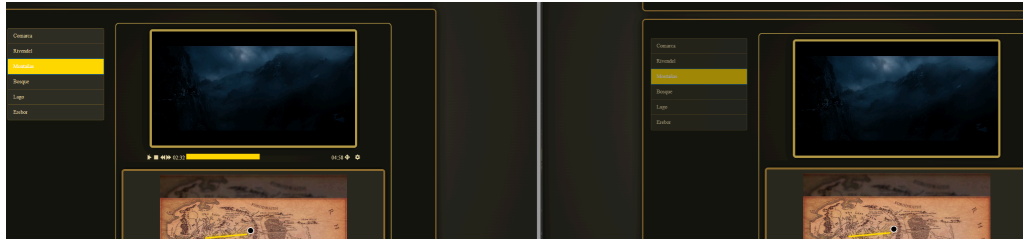
4.1 Sistema de conexión y señalización

Este módulo es responsable de establecer la comunicación inicial entre los participantes. Mediante WebSocket se intercambian las ofertas, respuestas y candidatos ICE necesarios para crear la conexión WebRTC.

```
let clients = [];  
  
ws.on('connection', (ws) => {  
  console.log("Cliente conectado");  
  clients.push({ ws, room: null, role: null });  
  
  ws.on('message', (message) => {  
    const data = JSON.parse(message);  
    console.log("Mensaje:", data.type);  
  
    switch(data.type) {  
      case 'join-room': {  
        const client = clients.find(c => c.ws === ws);  
        if (client) {  
          client.room = data.room;  
          client.role = data.role || null; // You, last week + practica?  
          console.log("Usuario unido a sala ${data.room} como ${client.role}");  
  
          const othersInRoom = clients.filter(other => other.ws !== ws && other.room === data.room);  
  
          if (othersInRoom.length > 0) {  
            ws.send(JSON.stringify({  
              type: 'peer-waiting',  
              room: data.room  
            }));  
          }  
  
          othersInRoom.forEach(other => {  
            if (other.ws.readyState === WebSocket.OPEN) {  
              other.ws.send(JSON.stringify({  
                type: 'peer-joined',  
                room: data.room  
              }));  
            }  
          });  
        }  
      }  
    }  
  });  
});
```

4.2 Sistema colaborativo de reproducción multimedia

Una vez establecida la conexión, el usuario controlador puede iniciar, pausar o desplazar la reproducción. Todas estas acciones se sincronizan automáticamente en el cliente remoto.



4.3 RTCDataChannel y comunicación interactiva

El RTCDataChannel constituye el principal mecanismo de intercambio de información en tiempo real. A través de él se transmiten eventos multimedia, mensajes de control y estados compartidos.

```
channel.onmessage = (event) => {
  const data = JSON.parse(event.data);

  // comandos
  if (data.type === "play") {
    video.play();
  }
  if (data.type === "pause") {
    video.pause();
  }
  if (data.type === "seek") {
    video.currentTime = data.time;
    moverZona(data.zona);
    actualizarRuta(data.zona);
    marcarZonaActiva(data.zona);
  }
  if (data.type === "quiz") {
    video.pause();
    mostrarQuiz(data.zona);
  }
  if (data.type === "command") {
    console.log("Comando recibido:", data.intent, data.text);
    executeCommandIntent(data.intent, data.text);
  }
  if (data.type === "respuestaQuiz") {
    quizActivo = false;
    ocultarEsperaQuiz();
    respuestasUsuario[data.zona] = {
      pregunta: data.pregunta,
      respuestaUsuario: data.respuestaUsuario,
      respuestaCorrecta: data.respuestaCorrecta,
      correcta: data.correcta
    };
  }
};
```

4.4 Sistema de cuestionarios sincronizados

Cuando el vídeo alcanza determinados puntos temporales, el sistema detiene automáticamente la reproducción y muestra preguntas configuradas previamente. Las respuestas quedan registradas y sincronizadas entre los participantes.

```
Mensaje P2P: {"type":"respuestaQuiz","correcta":false,"zona":"comarca","pregunta":"¿Qué objeto encuentra Bilbo que lo hace invisible?","respuestaUsuario":"El Mithril","respuestaCorrecta":"El Anillo"}  
Alumno respondió
```

4.5 Sistema de salas colaborativas

Las salas permiten gestionar múltiples sesiones simultáneas. Cada sala mantiene sus propios usuarios, estado multimedia y flujo de comunicación independiente.

4.6 Sistema de comandos inteligentes

Este módulo permite interpretar órdenes expresadas en lenguaje natural. Mediante Transformers.js se analizan los textos introducidos por el usuario y se traducen en acciones concretas sobre el reproductor multimedia.

Ejemplos:

- "Pausa el vídeo"
- "Avanza treinta segundos"
- "Reanuda la reproducción"

5. Tecnologías utilizadas

- Node.js (Servidor)
- WebSocket (Señalización)
- WebRTC (Comunicación P2P)
- RTCDataChannel (Intercambio de eventos)
- Transformers.js (IA local)
- HTML5 Video API
- CSS / Semantic UI React